DevBank

Microservices CI/CD on AWS

Capstone Project — Project 3

Submitted: June 07, 2026

Implementing Microservices with CodePipeline and ECS

1. Objective & Overview

This capstone project demonstrates the end-to-end implementation of a containerized microservices architecture for **DevBank**, a financial technology company delivering digital banking services across North America. The project showcases modern DevOps practices by automating CI/CD workflows, containerizing applications with Docker, and deploying them on a scalable, serverless infrastructure using Amazon ECS with AWS Fargate.

Project Goals

- Automate the complete CI/CD pipeline from code commit to production deployment
- Containerize both the NodeJS backend API and React frontend application using Docker
- Deploy microservices on Amazon ECS using serverless AWS Fargate compute
- Ensure high availability and seamless automated updates with zero manual intervention
- Validate the end-to-end pipeline by testing live service endpoints

Project Details

Project Title	Implementing Microservices with CodePipeline and ECS
Scenario	DevBank — Digital Banking Platform
Backend Stack	NodeJS (Express) — REST API on port 3000
Frontend Stack	React — served via nginx on port 80
CI/CD Tool	AWS CodePipeline + AWS CodeBuild
Container Registry	DockerHub
Deployment Target	Amazon ECS with AWS Fargate (serverless)
Source Control	GitHub (devbank-backend, devbank-frontend)
AWS Region	us-east-1 (N. Virginia)

2. Architecture Description

The solution follows a modern microservices architecture with two independent services — backend and frontend — each with its own dedicated CI/CD pipeline, Docker image, and ECS service. Both pipelines are fully automated and trigger on every GitHub push to the main branch.

CI/CD Pipeline Flow

Each service follows the same three-stage pipeline:

Stage	AWS Service	Action	Output
Source	GitHub + CodeConnections	Detects push to main branch, pulls source code	Source artifact in S3
Build	AWS CodeBuild	Runs buildspec.yml: docker build, docker push to DockerHub	imagedefinitions.json in S3
Deploy	Amazon ECS	Pulls new image from DockerHub, updates Fargate task	Running container on public IP

AWS Services Used

AWS Service	Purpose
AWS CodePipeline	Orchestrates the full CI/CD pipeline across all stages
AWS CodeBuild	Builds Docker images and pushes to DockerHub
Amazon ECS (Fargate)	Runs containerized services without managing servers
Amazon S3	Stores pipeline artifacts between stages
AWS Secrets Manager	Securely stores DockerHub credentials
AWS IAM	Controls permissions for all pipeline components
Amazon CloudWatch	Captures container logs for monitoring and debugging
Amazon VPC	Provides network isolation and public subnet routing
GitHub	Source code repository and pipeline trigger
DockerHub	Container image registry for built images

Infrastructure Design

Both services are deployed inside a single ECS cluster (**devbank-cluster**) running on AWS Fargate. Each service runs in a public subnet with auto-assigned public IP addresses, protected by a shared security group (**devbank-ecs-sg**) that allows inbound traffic on ports 3000 (backend) and 80 (frontend). IAM roles enforce least-privilege access across all pipeline components.

3. Step-by-Step Implementation

Step 1 — GitHub Repositories

Created two separate GitHub repositories: devbank-backend (NodeJS Express API) and devbank-frontend (React application). Each repository contains the application source code, a Dockerfile for containerization, a buildspec.yml for CodeBuild instructions, and supporting configuration files.

- devbank-backend: server.js, package.json, Dockerfile, buildspec.yml
- devbank-frontend: src/App.js, src/index.js, public/index.html, package.json, Dockerfile, nginx.conf, buildspec.yml

Step 2 — IAM Roles

Created three IAM roles with least-privilege permissions to allow each AWS service to perform only its required actions:

- CodeBuildDevBankRole — trusted by CodeBuild; allows ECR, CloudWatch, Secrets Manager, S3 access
- CodePipelineDevBankRole — trusted by CodePipeline; allows CodeBuild, ECS, S3, and iam:PassRole
- ecsTaskExecutionRole — trusted by ECS Tasks; allows image pulling and log writing

Step 3 — Secrets Manager

Stored DockerHub credentials securely in AWS Secrets Manager under the path devbank/dockerhub with two key/value pairs: username and password. The buildspec.yml files reference these secrets at build time so credentials are never stored in source code.

Step 4 — S3 Artifact Bucket

Created an S3 bucket (devbank-pipeline-artifacts) with versioning enabled to store pipeline artifacts — specifically the imagedefinitions.json file passed from the Build stage to the Deploy stage.

Step 5 — CodeBuild Projects

Created two CodeBuild projects (devbank-backend-build and devbank-frontend-build). Both use Amazon Linux 2023 with Privileged mode enabled to allow Docker daemon access inside the build container. Each project reads its buildspec.yml from the connected GitHub repository.

Step 6 — ECS Cluster and Task Definitions

Created a single ECS cluster (devbank-cluster) using the AWS Fargate serverless launch type. Defined two task definitions — devbank-backend-task (port 3000) and devbank-frontend-task (port 80) — each with 0.25 vCPU and 0.5 GB memory, CloudWatch logging configured.

Step 7 — Security Group and ECS Services

Created a security group (devbank-ecs-sg) allowing inbound traffic on ports 3000 and 80. Created two ECS services (backend-service and frontend-service) inside devbank-cluster, each running 1 Fargate task with auto-assigned public IP enabled.

Step 8 — CodePipeline

Created two CodePipelines (devbank-backend-pipeline and devbank-frontend-pipeline), each with three stages: Source (GitHub), Build (CodeBuild), and Deploy (Amazon ECS). Pipelines trigger automatically on every push to the main branch via GitHub webhook.

4. Challenges & Solutions

IAM Role — CodePipeline Use Case Not Found

Problem: The AWS IAM console does not list CodePipeline as a pre-defined use case when creating a role.

Solution: Selected 'Custom trust policy' and manually entered the trust policy JSON with codepipeline.amazonaws.com as the trusted service principal.

Webhook Creation Failed — Access Denied to CodeConnections

Problem: CodeBuild returned an 'Access denied' error when attempting to create a GitHub webhook via the CodeConnections ARN.

Solution: Added an inline IAM policy (CodeConnectionsPolicy) to both CodeBuildDevBankRole and CodePipelineDevBankRole granting codeconnections:UseConnection and codestar-connections:UseConnection permissions on the specific connection ARN.

S3 PutObject Access Denied During Pipeline Source Stage

Problem: CodePipeline failed at the Source stage with a 403 AccessDenied error when attempting to upload source artifacts to the S3 bucket.

Solution: Added an inline S3 policy (S3ArtifactPolicy) to CodePipelineDevBankRole and CodeBuildDevBankRole granting s3:GetObject, s3:PutObject, s3:ListBucket, and related permissions scoped to the artifacts bucket ARN.

ECS Circuit Breaker — CannotPullContainerError

Problem: The ECS frontend-service triggered the deployment circuit breaker because the Docker image ienglao/devbank-frontend:latest did not exist on DockerHub yet — the pipeline had not run its first successful build.

Solution: Proceeded to create and run CodePipeline first. The pipeline built and pushed the real Docker image to DockerHub, which automatically resolved the ECS CannotPullContainerError on the next deployment.

Frontend Build Failure — npm install exit code 254

Problem: The CodeBuild frontend build failed because package.json was missing from the devbank-frontend GitHub repository.

Solution: Added package.json with React dependencies to the repository root, restructured the project with src/ and public/ directories as required by Create React App, and added the missing src/index.js and public/index.html files.

Frontend Build Failure — index.js Not Found in /app/src

Problem: After adding package.json, the Docker build failed because src/index.js was not present in the repository.

Solution: Created src/index.js with the React DOM render entry point and committed it directly to the devbank-frontend GitHub repository via the web UI, which automatically triggered a new pipeline run.

---

5. Conclusion

This capstone project successfully demonstrates the implementation of a fully automated, production-grade CI/CD pipeline for a microservices-based banking application on AWS. By integrating GitHub, AWS CodePipeline, AWS CodeBuild, Docker, and Amazon ECS Fargate, the project achieves a seamless delivery workflow where every code push automatically results in a running, updated container in the cloud — with no manual server management required.

Key Outcomes

- Two fully automated CI/CD pipelines operational for backend and frontend services
- Docker images automatically built and pushed to DockerHub on every GitHub commit
- Both microservices running as live Fargate tasks accessible via public IP addresses
- Backend API verified at http://:3000 returning JSON health and account data
- Frontend React application verified at http://3.93.23.61:80 rendering in the browser
- All AWS resources secured with least-privilege IAM roles and Secrets Manager
- Six real-world challenges encountered and independently resolved during implementation

Skills Demonstrated

Category	Skills
AWS CI/CD	CodePipeline, CodeBuild, buildspec.yml, artifact management
Containerization	Docker multi-stage builds, Dockerfile authoring, DockerHub
Cloud Infrastructure	ECS Fargate, task definitions, services, VPC, security groups
Security	IAM roles, inline policies, Secrets Manager, least privilege
Troubleshooting	CloudWatch logs, ECS stopped task analysis, IAM debugging
Source Control	GitHub repositories, webhook triggers, branch-based deployments